Deep Q-Network (DQN) on Atari Pong: Conclusions and Analysis

CSCN 8020 - Reinforcement Learning Programming Assignment 3 - Final Report

---

Executive Summary

This report presents the implementation and experimental evaluation of a Deep Q-Network (DQN) agent trained to play Atari Pong. The agent successfully demonstrates learning behavior through experience replay, target network stabilization, and epsilon-greedy exploration. Key experiments systematically investigated the impact of batch size and target network update frequency on learning performance.

Network Architecture

The final DQN architecture consists of a convolutional neural network (CNN) designed to process high-dimensional visual input:

- **Input Layer:** Stack of 4 consecutive grayscale frames (84×80 pixels each), resulting in shape (4, 84, 80)
- **Convolutional Layer 1:** 32 filters, 8×8 kernel, stride 4, ReLU activation
- **Convolutional Layer 2:** 64 filters, 4×4 kernel, stride 2, ReLU activation
- **Convolutional Layer 3:** 64 filters, 3×3 kernel, stride 1, ReLU activation
- **Fully Connected Layer 1:** 512 units, ReLU activation
- **Output Layer:** 6 units (one per action), linear activation

This architecture enables the agent to extract spatial features (paddle position, ball trajectory) from raw pixel observations and learn effective action-value mappings without manual feature engineering.

Experimental Results

Training Duration Analysis (5, 250, 500 Episodes)

Training duration significantly impacts DQN performance in complex environments like Pong:

5-episode run (debug validation):

- Scores remain consistently around -20 (maximum loss margin)
- No observable learning or improvement
- Serves only to verify implementation correctness

250-episode run (partial learning):

- Gradual improvement emerges after approximately 150-200 episodes
- Moving average reward improves from -20 to roughly -17 to -16 range
- Learning trend is visible but unstable

500-episode run (meaningful learning):

- Clear upward trend after approximately 300 episodes
- Scores occasionally improve to -14 to -12 range, with some episodes reaching -9 or better
- Higher variability present, indicating active exploration alongside exploitation
- Agent demonstrates reduced loss margins but has not yet achieved consistent winning performance

Key insight: While 500 episodes show substantially better performance than shorter runs, full convergence to a competitive Pong policy requires additional training time, likely 1000+ episodes or further hyperparameter optimization.

Experiment 1: Batch Size Comparison (8 vs 16)

Both batch sizes were evaluated over 250 episodes with all other hyperparameters held constant (target update every 10 episodes).

Batch size 8 (default):

- Moving average reward improves from -20 to approximately -17 by episode 250
- Moderate variability in episode scores
- Gradual, steady learning trend

Batch size 16:

- Moving average reward shows similar final performance around -17 to -16
- Slightly smoother learning curve due to lower-variance gradient estimates
- Comparable or marginally better performance by end of training

Conclusion: Under the 250-episode training budget, batch size 16 provides marginally smoother convergence but does not demonstrate a substantial performance advantage over batch size 8. Both configurations achieve similar final performance levels.

Experiment 2: Target Network Update Frequency (Every 3 vs 10 Episodes)

Target network synchronization frequency was tested over 250 episodes with batch size fixed at 8.

Update every 10 episodes (default):

- Moving average reward improves from -20 to approximately -16 to -15 range
- Stable, gradual learning progression
- Lower instability in Q-value targets

Update every 3 episodes:

- Moving average reward shows more erratic behavior
- Final performance slightly worse, around -17 to -16

- More frequent updates introduce instability due to rapidly changing targets
- Q-value estimates less stable when target network updates too frequently

Conclusion: Updating the target network every 10 episodes yields more stable and better overall performance compared to every 3 episodes. Frequent target updates reduce the stabilization benefit of the target network mechanism, leading to noisier learning.

Best Hyperparameter Combination

Based on experimental evidence across 250-episode training runs:

Recommended configuration:

- **Batch size:** 8 (default)
- **Target network update frequency:** Every 10 episodes (default)
- **Discount factor γ:** 0.95
- **Exploration parameters:** $\epsilon_{\text{init}} = 1.0$, decay $= 0.995$, $\epsilon_{\text{min}} = 0.05$

Justification:

- Batch size 8 provides efficient gradient updates without requiring excessive memory or computation, while batch size 16 offers only marginal smoothing benefits under this training budget.
- Target update every 10 episodes maintains stable Q-value targets, preventing the instability observed with more frequent updates (every 3 episodes).
- This combination balances learning stability, sample efficiency, and computational cost effectively.

Key Observations

Learning Dynamics

1. **Sparse Rewards:** Pong provides delayed, sparse rewards (only at episode end), making early learning slow and requiring substantial exploration.
2. **Epsilon Decay:** Gradual reduction of exploration rate enables the agent to transition from random behavior to exploitation of learned policies over time.
3. **Experience Replay:** Random sampling from the replay buffer breaks temporal correlations and stabilizes training by providing diverse mini-batches.
4. **Target Network Stabilization:** Periodic synchronization prevents rapidly shifting Q-value targets, which is critical for convergence in deep RL.

Performance Variability

Despite clear learning trends, reward curves remain noisy throughout training:

- Reinforcement learning in stochastic environments inherently exhibits high variance
- Occasional performance drops are expected and do not indicate failure
- Moving averages (last 5 episodes) provide clearer insight into underlying learning trends

Convergence Status

By episode 500, the agent demonstrates meaningful improvement (scores improving from -21 to occasional -9 or better), but:

- Has not yet achieved consistent positive scores (winning games)
- Requires extended training (likely 1000-2000 episodes) or further tuning to reach competitive performance
- Current performance indicates learning is occurring but convergence is incomplete

---

Limitations and Future Improvements

Current Limitations

- **Training Duration:** 500 episodes insufficient for full convergence in Pong
- **Hyperparameter Search:** Limited exploration of learning rate, replay buffer size, and network architecture variations
- **Reward Engineering:** No reward shaping applied; agent relies solely on sparse game outcomes

Potential Enhancements

1. **Extended Training:** Run 1000-2000 episodes to observe full convergence behavior
2. **Learning Rate Tuning:** Experiment with adaptive learning rate schedules
3. **Prioritized Experience Replay:** Sample important transitions more frequently to improve sample efficiency
4. **Double DQN:** Reduce overestimation bias in Q-value updates
5. **Dueling DQN:** Separate value and advantage streams for better generalization
6. **Hyperparameter Grid Search:** Systematically evaluate combinations of batch size, learning rate, and update frequency

---

Conclusion

This project successfully implemented and evaluated a Deep Q-Network agent for Atari Pong, demonstrating the fundamental components of deep reinforcement learning: convolutional feature extraction, experience replay, target network stabilization, and epsilon-greedy exploration.

Experimental results confirm that:

- **Training duration** is the dominant factor in achieving good performance, with 500 episodes showing clear improvement over 250 but still requiring further training for mastery
- **Batch size 8** and **target network update every 10 episodes** provide the best balance of stability and performance under the

tested configurations
- **DQN mechanisms** (replay buffer, target network) are essential for stable learning in high-dimensional visual environments

The agent progresses from random play (scores around -21) to improved performance (occasional scores around -9 to -12 by episode 500), validating the effectiveness of the DQN algorithm. Continued training and potential architectural enhancements would enable the agent to achieve competitive or winning performance in Pong.

Overall, this assignment provides practical insight into the challenges and successes of applying deep reinforcement learning to complex control tasks with high-dimensional state spaces.